

실측으로 원인을 찾는 개발자

"답이 없는 문제는 없다"

문제를 해결함에 있어 **체계적**으로 정리해 원인을 찾아내는 것은 개발자가 가져야 하는 역량입니다. 항상 문제에 직면했을 때, **실측**으로 원인을 확인하고 구조를 개선했을 때 정답에 가까워지는 **경험**을 했습니다.

WEB FRONTEND DEVELOPER

김도혁 · Kim Do Hyuk

PORTFOLIO · 2026

01. ABOUT ME

✉ hyeooooook@gmail.com

☎ 010-9400-6992

🌐 hyeooooook.tistory.com

🐙 github.com/DoHyuk-Centric

변경된 블록만 다시 렌더링합니다.

- 1 마크다운 미리보기를 변경 블록 단위 렌더링으로 개선해 성능을 최대 8.2배 향상시켰습니다.

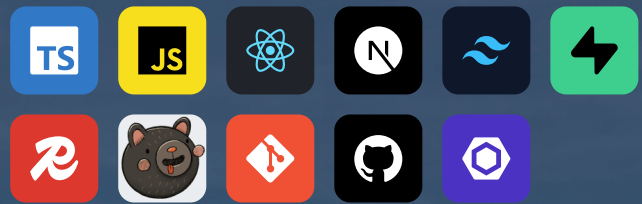
서버 상태를 분리해 요청을 줄였습니다.

- 2 관리자 금칙어 관리에 TanStack Query를 도입해 GET 요청을 18회에서 3회로 감소시켰습니다.

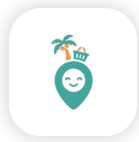
사용자 패턴으로 점수 공식을 검증했습니다.

- 3 신뢰도·인기 게시물 점수 공식을 설계하고 9가지 사용자 패턴을 시뮬레이션해 등급 분포를 사전 검증했습니다.

기술 스택

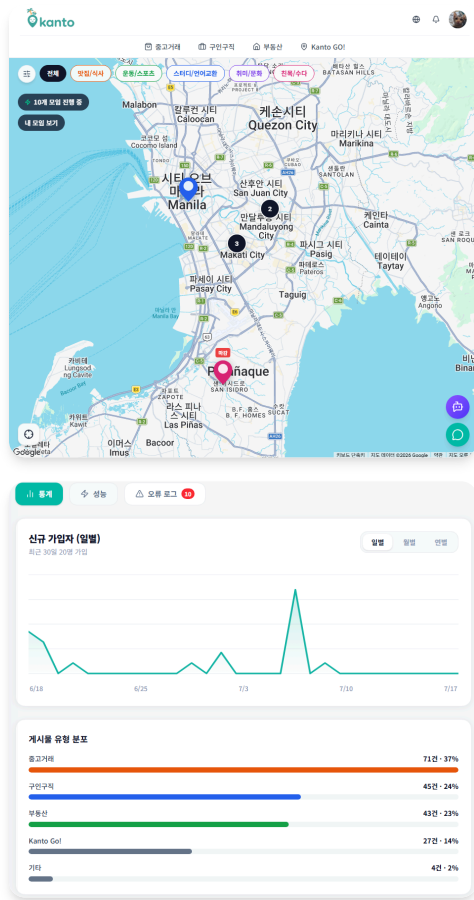


02. PROJECT - KANTO · DETAILS (1/5)



Kanto

| 필리핀 한인을 위한 통합 생활 플랫폼



개요

- 필리핀은 "세계 소셜 미디어 수도"로 꼽힐 만큼 디지털 이용률이 높지만, 중고거래·구인구직·부동산·모임 같은 생활 정보는 전용 플랫폼 없이 여러 페이스북 그룹에 흩어져 있었습니다.
- 필리핀 한인들이 신뢰할 수 있는 통합 창구가 없다는 문제를 발견했습니다.
- Kanto("우리 동네"라는 뜻)는 이 정보들을 하나의 플랫폼으로 모아, 더 빠르고 안전하게 생활 정보를 찾을 수 있도록 만든 통합 생활 플랫폼입니다.

핵심 기능

- 카테고리별 게시판** - 중고거래·구인구직·부동산·모임 게시글을 한 플랫폼에서 등록·검색·관리
- 사진 검열** - AI로 부적절한 이미지를 자동 필터링해 게시글 품질 관리
- Kanto Go** - 지도 기반 실시간 번개모임 매칭
- 1:1 채팅·신뢰도** - Realtime 채팅과 칸토 망고 점수(KTS-KPPS)로 안전한 거래 환경 제공
- AI 챗봇** - Groq·Gemini·Cerebras 멀티 LLM 풀백으로 안정적으로 응답하며, 반복 문의를 자동 처리해 운영 부담 절감

프로젝트 정보

개발 인원	팀 프로젝트 · 4인
플랫폼	Web
개발 기간	2026.05 - 2026.07
결과	파이널 1위
관리자 페이지	kanto-iota.vercel.app/admin
테스트 계정	asdf1234@naver.com asdf1234

개발 환경

언어	TypeScript
프레임워크	Next.js 16 · React 19 · shadcn/ui · Tailwind CSS
DB	PostgreSQL(Supabase)
보안	Redis(Upstash) 레이트리밋
라이브러리	Zustand · TanStack Query · next-intl
AI/LLM	Groq · Cerebras · Gemini · OpenAI
API	Supabase · Xendit · Notion API · MyMemory · Google Maps · Nodemailer(Gmail SMTP)
배포/인프라	Vercel · GitHub Actions(CI/CD)
모니터링	Sentry

경험

프로필 페이지 · 칸토 망고 점수

당근은도처럼 신뢰가 핵심인 서비스엔 신뢰도를 시각적으로 보여줄 지표가 필요하다고 판단해 **KTS(신뢰도)·KPPS(인기도)** 점수 시스템을 설계했습니다.

• KTS 공식

$$KTS = \text{clamp}(36 + \text{Review} + \text{Transaction} + \text{Activity} + \text{Trend} - \text{Penalty}, 0, 100)$$

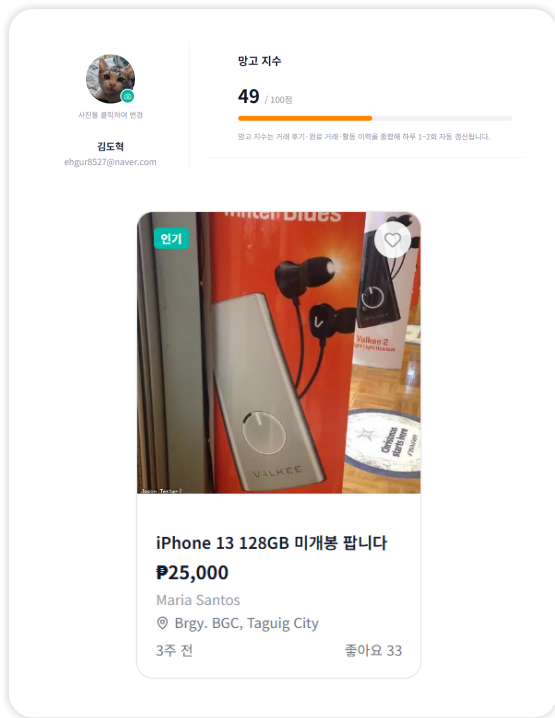
Review 리뷰 평점 · Transaction 거래 완료 · Activity 활동 · Trend 등급 추이 · Penalty 신고·제재

• KPPS 공식

$$KPPS = \text{clamp}(\text{Engagement} + \text{LikeRate} + \text{KTSBonus} + \text{Quality} + \text{Recency} - \text{Report}, 0, 100)$$

Engagement 좋아요·조회수 · LikeRate 좋아요 전환율 · KTSBonus 신뢰등급 · Quality 이미지·정보 완성도 · Recency 최신성 · Report 신고

- **9가지 유저 시나리오로 시뮬레이션해** 등급 분포(A 7%·B 22%·C 45%·D 22%·E 4%)를 사전 검증한 뒤 기준 확정
- 작성자가 E등급이면 위험수준 유저라고 판단해 순위·점수와 무관하게 **배지 제외**
- 점수는 매일 일정 시간 **pg_cron**으로 자동 재계산



채팅 · 관리자 채팅 로그

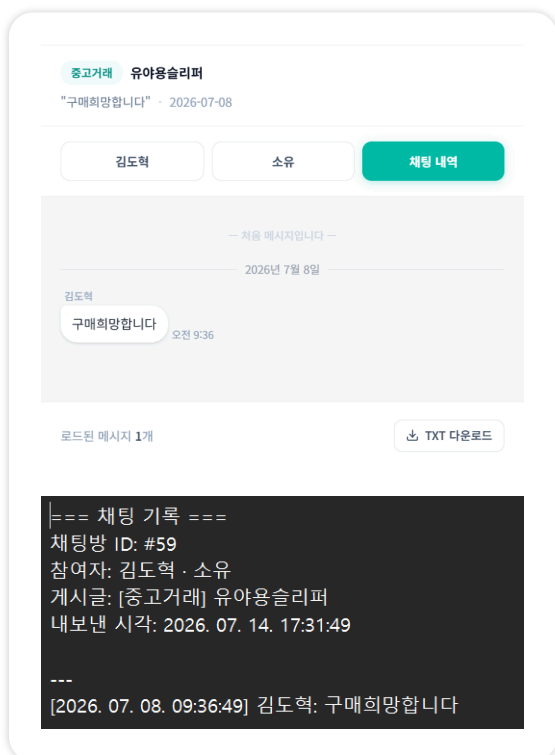
유저 간의 소통을 위해 **Supabase Realtime 기반 1:1 채팅**을 전체 구현하고, 운영적 측면에서 생각해 법적 분쟁·사용자 문의에 대응할 수 있는 관리자 도구를 구축했습니다.

사용자

- Realtime 메시지 송수신·읽음 처리에 **읽지 않은 메시지** 구현
- 언마운트 시 리셋 + 낙관적 업데이트 적용
- N초 내 N회 이상 전송 시 **도배 방지** 쿨다운 적용(관리자 운영 관리에서 수정 기능 연동)

관리자

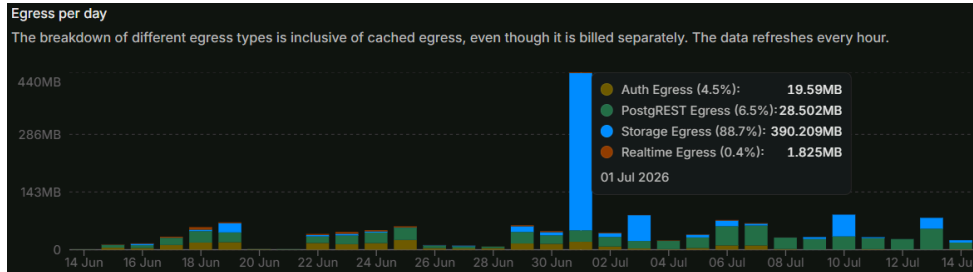
- 관리자가 신고된 채팅방의 대화 내역을 **페이지네이션으로 조회**(최대 20개씩 로드)
- 대화 내역을 **TXT로 다운로드**해 분쟁 발생 시 증빙 자료로 활용
- 대화방을 나가거나 회원을 탈퇴하더라도 일정 기간동안은 내역을 유지



문제해결

Supabase 트래픽 한도 초과 대응

기획 단계에서 무료 플랜의 트래픽·토큰 한도를 미리 계산해 소진 시나리오와 대응 전략을 세워줬지만, 실제로는 예상 밖의 지점에서 문제가 터졌습니다.

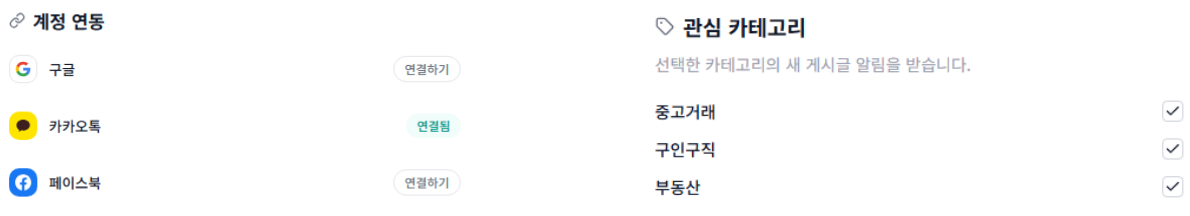


7/1 Storage Egress 390MB(전체 88.7%) 급증 → WebP 전환 후 재발 없음

- 이미지를 **WebP로 사전 전환**해두지 않은 상태에서, 시연 준비 중 봇 계정과 목업 이미지를 대량 등록하다 **Supabase 트래픽 한도 초과**가 발생했습니다.
- 사전에 세워둔 대응 전략은 있었지만 트래픽에서 문제가 터질 것이라곤 생각하지 못했습니다. 하지만 과도한 트래픽은 최근 추가한 봇과 게시글이 원인일 것으로 추정해 **이미지 용량**으로 특정했습니다.
- DB 양이 많아 부계정 이전 대신 **유료 플랜 전환**으로 즉시 정상화하고, 업로드 단계 WebP 변환 로직을 추가해 동일 조건으로 이미지를 올려도 **390MB → 64MB(약 84% 감소)**로 트래픽이 안정적임을 확인해 재발 방지까지 검증했습니다.

내 프로필 페이지 - 서버/클라이언트 컴포넌트 분리

프로필 페이지를 처음 구현할 때는 로그인·회원가입 페이지가 아직 구현되기 전이라, 빠른 개발을 위해 목업 데이터를 쓰는 클라이언트 컴포넌트로 만들었습니다. 이후 Supabase API로 실제 데이터를 연결하고 소셜 로그인 "연결하기/연결됨" 버튼과 관심 카테고리를 구현하는 과정에서, 깜빡이는 **CLS(레이아웃 시프트)** 현상이 생겼습니다.

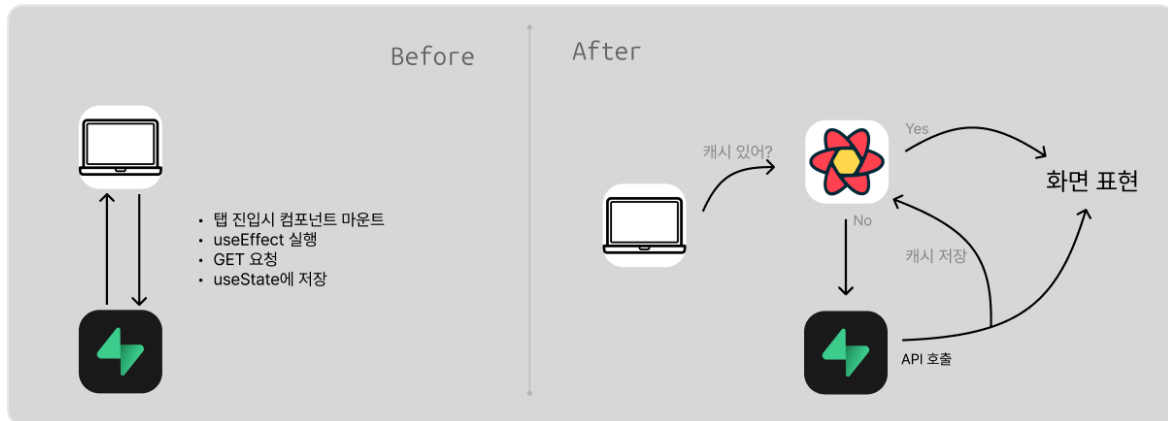


- 원인은 각 섹션이 **클라이언트 쪽에서 자체적으로 데이터를 불러오던 구조**였기 때문이었습니다. 페이지가 먼저 빈 상태로 그려지고, 그 다음 브라우저에서 데이터를 요청해오니 버튼·카테고리가 늦게 나타난 것이었습니다.
- 팀 컨벤션에 따라 페이지 전체를 서버 컴포넌트로 두고**, 필요한 데이터를 서버에서 미리 불러와 완성된 상태로 클라이언트에 내려주는 구조로 전환했습니다. 클라이언트 혹은 자체 fetch 로직을 걷어내고, 서버가 준 초기값만 받아쓰도록 단순화했습니다.
- 이후 페이지가 복잡해지면서(리뷰·모임·활동 통계 등) 늘어난 기능들도 같은 원칙을 유지해, 데이터 조회는 서버 컴포넌트에 모으고 인터랙션만 클라이언트 컴포넌트로 넘기는 구조를 끝까지 지켰습니다.

문제해결 (2)

관리자 콘텐츠 중복 GET 제거 - TanStack Query 캐싱

금치어스팸 탭을 전환할 때 컴포넌트가 언마운트되며 useEffect가 다시 실행돼, 이미 조회한 서버 데이터를 반복 요청하고 있었습니다.



- 서버 조회값을 useState에 두면 탭 언마운트와 함께 사라져, 재진입할 때마다 useEffect가 같은 GET을 반복했습니다. 조회 데이터는 컴포넌트 수명과 분리해 TanStack Query가 관리하고, draft-editingId처럼 화면에만 필요한 상태만 useState에 남겼습니다.
- 설정·템플릿 수정 결과는 캐시에 즉시 반영해 추가 GET을 막고, 금치어 CRUD 후에는 해당 목록 캐시만 무효화해 최신 데이터를 다시 조회했습니다.
- 탭 5회 왕복 + 롤 등록/목록 2회 왕복을 동일하게 재현해 GET 요청이 **18회 → 3회(약 83.3% 감소)**로 줄어든 것을 네트워크 탭에서 확인했습니다.

로그인 무차별 대입 공격 방지 - Redis(Upstash)

로그인 API에 시도 횟수 제한이 없어 무차별 대입 공격에 취약하다는 문제를 코드 리뷰 중 발견했습니다.



- 서버 인메모리 카운터로 막으려 했지만, **서버리스 환경은 인스턴스 간 상태가 공유되지 않아** 단순 카운터로는 우회가 가능하다는 걸 확인했습니다.
- **Upstash Redis**에 IP별 로그인 실패 횟수를 기록하도록 설계해, 인스턴스가 몇 개든 같은 카운터를 공유하도록 만들었습니다.
- **5회 이상 실패 시 15분간 로그인을 차단**하는 구조로 직접 설계·구현했습니다.

협업 방식

브랜치 전략 · 코드 리뷰 (Git Flow)

feature · fix · hotfix



develop



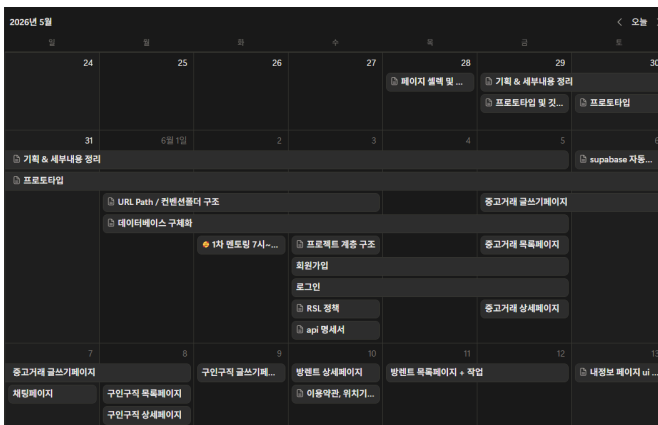
release



main (배포)

- 6종 브랜치(main·develop·release·feature·fix·hotfix)로 운영하고, 모든 병합은 본인 외 1명 이상 Approve를 받은 PR로만 진행했습니다.
- 3단계 점진 배포(6/22 · 7/1 · 7/6)로 릴리스했고, 결제 웹훅·Storage 트래픽 이슈는 hotfix 브랜치로 실서비스 중 대응했습니다.

일정 관리 (Notion WBS)



5/24~6/13

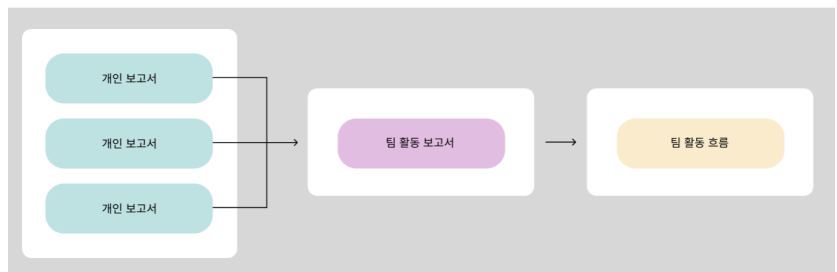


6/14~7/8

1~2주차에 인증·마켓 기본기를 다진 뒤, 3주차부터 거래 흐름이 안정되며 팀 전체가 관리자·운영 기능으로 무게 중심을 옮겼습니다. 4주차 Kanto Go 확장으로 기능이 늘자 5주차 계획을 전 구간 성능 감사로 조정했고, 6주차는 신규 기능 없이 안정화·검증에만 집중하도록 범위를 좁혔습니다.

주간 보고서 운영

매주 팀 전체 커밋 로그를 취합해 활동 보고서를 작성했습니다. 기여도 산정에는 Notion 회의록·자기보고 WBS는 물론 단순 커밋 개수도 왜곡 소지가 있다고 판단해 배제하고, 최초 작성자·코드 계보·PR 근거로 담당 영역과 주도 기능을 정리했습니다.



보고서 초안 작성에는 같은 AI(Claude)를 단계마다 다르게 프롬프팅했습니다. 팀원 개인 보고서 3건을 팀 활동 보고서로 합치는 프롬프트와, 이를 다시 팀 활동 흐름으로 요약하는 프롬프트를 나눠 순차 실행했습니다.

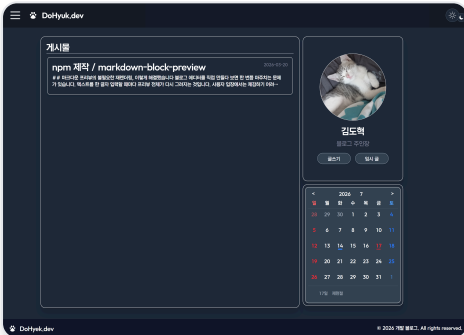
참고자료 (원본 PDF)

개인 보고서 팀 활동 보고서 팀원 기여도 팀 활동 흐름



개요

- 첫 프론트엔드 프로젝트입니다. 직접 관리 · 유지보수가 가능한 블로그가 필요했고, 당시 velog에서 이미지 버튼 클릭 시 커서 위치에 줄바꿈 없이 마크다운이 바로 삽입돼 문장 중간에 이미지 태그가 붙던 불편함을 직접 개선하고자 만들었습니다.
- 당시 다룰 수 있는 기술은 Vanilla JS였고, Vite의 멀티 페이지 빌드로 페이지마다 필요한 스크립트만 로드하는 MPA 구조를 직접 설계했습니다.
- About 페이지에 데스크탑 Windows XP·모바일 삼성 One UI 테마를 인터랙션으로 재현해, 프로젝트 소개 자체를 하나의 UI 실험으로 구성했습니다.



핵심 기능

- 마크다운 에디터 - 블록 단위 프리뷰로 8.2배 빠른 렌더링, 자체 npm 패키지 (markdown-block-preview)로 개발·배포
- 게시글 관리 - 작성·수정·삭제 CRUD, 임시저장·휴지통 상태 머신으로 실수 방지
- 이미지 지연 업로드 - blob 임시 주소로 미리보기, 발행 시점에만 실제 업로드해 고아 이미지 방지
- 소셜 로그인 - Supabase Auth 기반 GitHub·Google OAuth

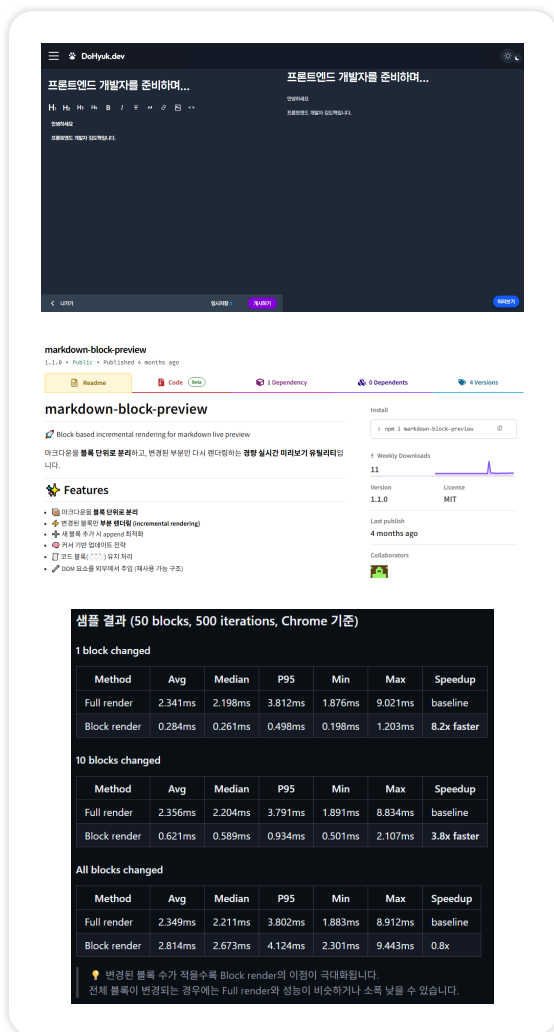
프로젝트 정보

개발 인원	개인 프로젝트
플랫폼	Web
개발 기간	2026.01 - 2026.03

개발 환경

언어	JavaScript
프레임워크	Vanilla JS · Vite(MPA)
스타일링	Tailwind CSS v4
DB	PostgreSQL(Supabase)
인증	Supabase Auth(GitHub·Google OAuth)
API	Edge Function(공공데이터포털 공휴일 프록시)
라이브러리	자체 npm 패키지(markdown-block-preview)
배포/인프라	Vercel · GitHub Actions(CI, gitleaks)

경험



The image shows two parts of the project. The top part is a screenshot of the application interface, which displays a markdown editor with a live preview. The bottom part is a screenshot of the npm page for the 'markdown-block-preview' package. The npm page includes the package name, version (1.1.4), and a description: 'Block-based incremental rendering for markdown live preview'. It also lists features such as 'Markdown to HTML block incremental rendering', 'Incremental rendering (incremental rendering)', 'Support for custom renderers', 'Support for custom renderers', 'Support for custom renderers', and 'Support for custom renderers'. Below the features, there is a table showing benchmark results for 1 block changed, 10 blocks changed, and all blocks changed. The table compares the 'Full render' method with the 'Block render' method, showing significant performance improvements in terms of average time, median time, P95, and speedup.

Method	Avg	Median	P95	Min	Max	Speedup
Full render	2.341ms	2.198ms	3.812ms	1.876ms	9.021ms	baseline
Block render	0.284ms	0.261ms	0.498ms	0.198ms	1.203ms	8.2x faster

markdown-block-preview · 자체 npm 패키지 ↗

마크다운 에디터에서 입력할 때마다 innerHTML을 통째로 교체하며 미리 보기 전체를 다시 그린다 보니, 글자 하나 칠 때마다 미리보기 영역이 통째로 깜빡거렸습니다.

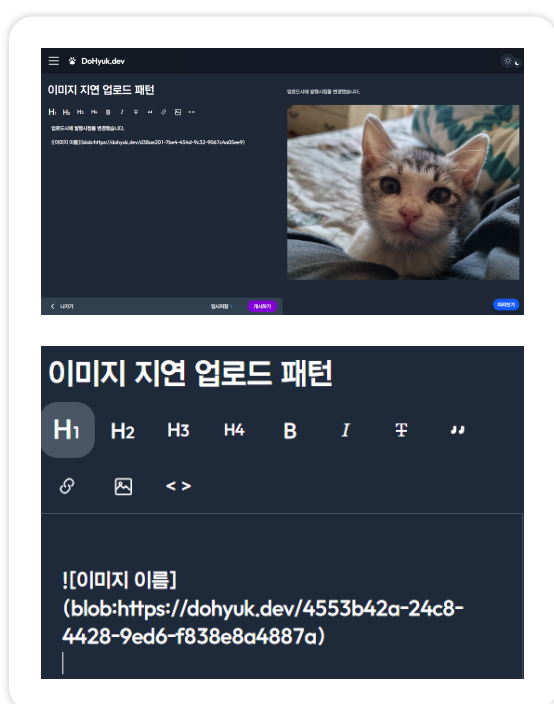
또한, 글이 길어질수록 재렌더링하는 영역이 많아 성능이 떨어질 것으로 판단했습니다.

원인 실측

- 전체 문서 재파싱 **0.98ms** vs 블록 1개만 파싱 **0.005ms** → 파싱은 병목이 아님
- 전체 innerHTML 반영 **13.45ms** vs 블록 1개만 반영 **0.18ms** → DOM 트리 재구성이 병목
- naive 방식(매번 전체 재파싱 + innerHTML 통째 교체)은 700블록부터 프레임 예산(16.6ms)을 초과

해결

- 빈 줄 기준으로 문서를 블록 단위로 나누고, 이전 블록과 비교해 **바뀐 블록만** DOM에 반영, 구조가 바뀌면 전체 재구성으로 풀백
- 마땅한 라이브러리가 없어 **별도 npm 패키지로 직접 개발·배포**해 이 프로젝트 글쓰기 에디터에 적용
- 문서 길이와 무관하게 편집당 **0.03~0.07ms(naive 대비 최대 8.2배 개선)** 유지



The image shows a screenshot of the application interface for the 'Image Upload Pattern'. It displays a markdown editor with a live preview. The preview shows a cat image and a URL: `(blob:https://dohyuk.dev/4553b42a-24c8-4428-9ed6-f838e8a4887a)`. The editor also shows the URL in the address bar.

이미지 지연 업로드 패턴

처음엔 이미지를 선택하는 즉시 Storage에 업로드했는데, 임시저장 없이 중간에 나가버리면 쓰이지 않는 이미지가 Storage에 그대로 쌓이는 문제가 있었습니다.

- 발행 시점에만** 실제 업로드하도록 바꿨는데, 그 전까지 미리보기는 어떻게 보여줄지가 새로운 문제였습니다.
- URL.createObjectURL()**로 만든 **blob: 임시 주소**로 해결했습니다 - 파일을 선택하면 브라우저가 그 파일을 가리키는 로컬 주소를 즉석에서 만들어 미리보기에 쓰고, 실제 파일은 별도 Map에 그 주소와 함께 잠깐 담아뒀습니다.
- 발행 버튼을 누른 시점에만 이 주소들을 찾아 실제로 업로드하고, 마크다운 속 주소를 진짜 업로드된 주소로 바꿔치기해 불필요한 업로드와 고아 이미지 발생을 줄였습니다.

문제해결

인증 수단 일원화 - Firebase → Supabase

게시글을 작성자가 아닌 사용자도 수정·삭제할 수 있는 보안 문제를 발견했습니다. 데이터(Supabase)와 인증(Firebase)이 서로 다른 공급자라 UID 체계가 달라, 소유권을 안전하게 검증할 방법이 없었습니다.

```
import { auth } from "../firebase";
import { onAuthStateChanged } from "firebase/auth";

loginState();

function loginState() {
  onAuthStateChanged(auth, (user) => {
    if (user) {
      localStorage.setItem("isLogin", "true");
    } else {
      localStorage.setItem("isLogin", "false");
    }
  });
}
```

```
import { supabase } from "../../src/supabase.js";

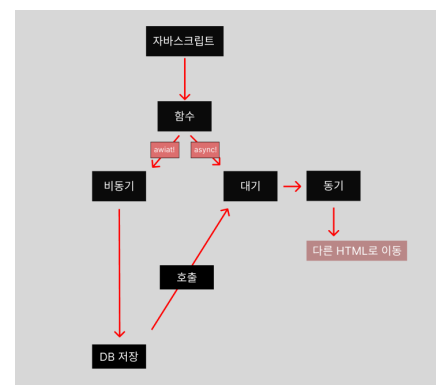
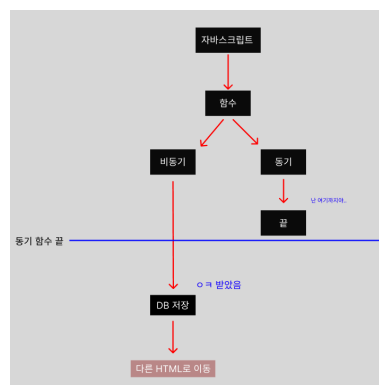
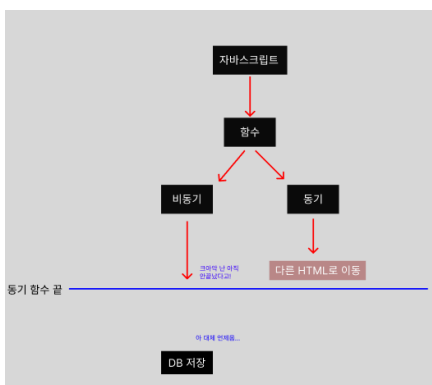
loginState();

function loginState() {
  supabase.auth.onAuthStateChanged((event, session) => {
    if (session?.user) {
      localStorage.setItem("isLogin", "true");
    } else {
      localStorage.setItem("isLogin", "false");
    }
  });
}
```

- UID 매핑 DB·JWT 로컬스토리지 저장 방식을 검토했지만, 이중 경유로 인한 성능 저하와 XSS 위험으로 기각했습니다.
- Firebase를 걷어내고 **Supabase Auth(GitHub·Google OAuth)**로 일원화해 인증·데이터를 하나의 공급자로 통합했습니다.
- **checkPostOwner**가 로그인 사용자 id와 게시글 **user_id**를 직접 비교해 **RLS**로 소유권을 강제하도록 검증까지 마쳤습니다.

비동기 저장 완료 전 페이지 이동 - 게시글 유실 버그

저장 버튼을 눌러도 에러 없이 목록 페이지로 이동해 "저장됐다"고 착각하기 쉬웠지만, 실제로는 글이 DB에 저장되지 않는 버그였습니다.



- **insert()** 요청의 응답을 기다리지 않고 곧바로 페이지를 이동시켜, 요청이 서버에 당기도 전에 실행 컨텍스트가 폐기됐습니다.
- 페이지 이동 호출만 지운 1차 수정은 증상만 없었을 뿐, **동기·비동기가 서로 무관하게 움직여** 실제로 완료를 기다리는 구조는 아니었습니다.
- **async/await**로 비동기 함수의 실행과 결과를 실제로 기다린 뒤에만 페이지를 이동하도록 구조화했고, 이후 DB 쓰기 전체에 같은 패턴을 적용했습니다.

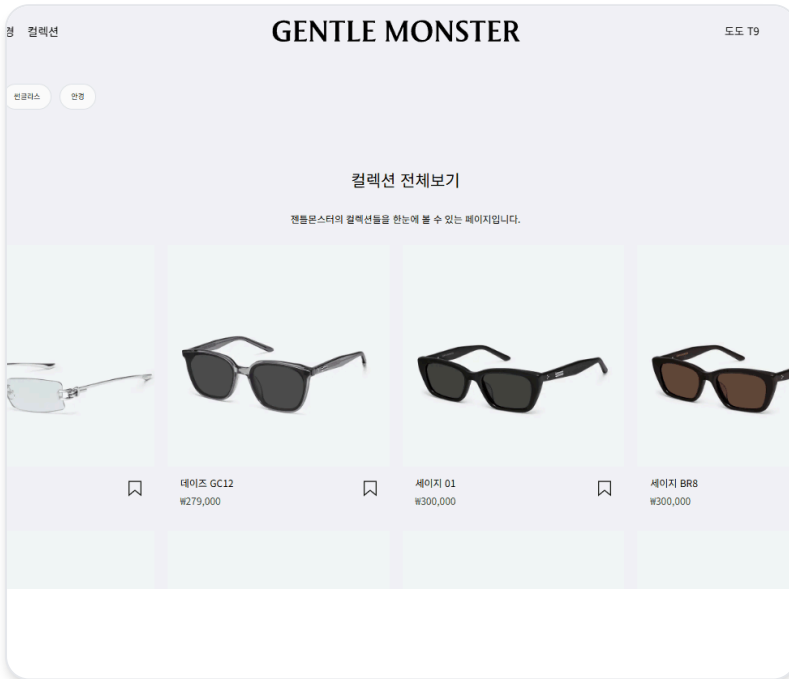


Gentlelion ↗

Vanilla JavaScript 기반 패션 커머스 · 유저/관리자 서비스



유저·관리자·공통 API를 하나의 모노레포로 분리했습니다.



구조 설계

- Vite Workspace로 user·admin·share-api 패키지 분리
- 유저와 관리자 앱을 각각 빌드해 두 서비스로 개별 배포
- 공통 API 계층에서 인증 토큰 주입·에러 처리·HTTP 메서드 관리

담당 기능

- 상품 조회·필터·정렬, 장바구니 수량 변경, 주문 생성
- 관리자 사용자 목록·삭제, 주문 목록·상세 조회
- 공통 컴포넌트와 브랜치 전략, 이슈 기반 작업 흐름 관리

프로젝트 정보

개발 인원	3인 팀 프로젝트 · 팀장
개발 기간	2026.03.26 - 2026.04.20
배포	유저 · 관리자 서비스 분리

개발 환경

프론트엔드	HTML · CSS · JavaScript
빌드/구조	Vite · npm Workspaces
테스트	Playwright

핵심 5개 사용자 흐름을 E2E로 검증했습니다.

The screenshot shows a web application for 'GENTLE MONSTER' with a sidebar menu containing '대시보드', '사용자 관리', '제품 관리' (selected), '주문 관리', and '로그아웃'. The main content area is titled '제품 관리' and displays a table of products.

제품 ID	이름	카테고리	가격	재고	작업
173	Playwright 테스트 상품	선글라스	₩290,000	10개	✎ 🗑
172	Playwright 테스트 상품	선글라스	₩290,000	10개	✎ 🗑
171	Playwright 테스트 상품	선글라스	₩290,000	10개	✎ 🗑
170	Playwright 테스트 상품	선글라스	₩290,000	10개	✎ 🗑
163	test	collection	₩290,000	100개	✎ 🗑
162	진 BRC9	안경	₩289,000	10개	✎ 🗑
161	Ep 02	안경	₩330,000	10개	✎ 🗑
160	분조	안경	₩320,000	9개	✎ 🗑
159	오즈	안경	₩320,000	10개	✎ 🗑
158	가우스 GC9	안경	₩320,000	10개	✎ 🗑
157	론 BRC15	안경	₩320,000	10개	✎ 🗑

유저와 관리자 양쪽에서 CRUD가 이어지면서 수정 한 번이 다른 흐름을 깨뜨리는지 수동으로 확인하기 어려웠습니다.

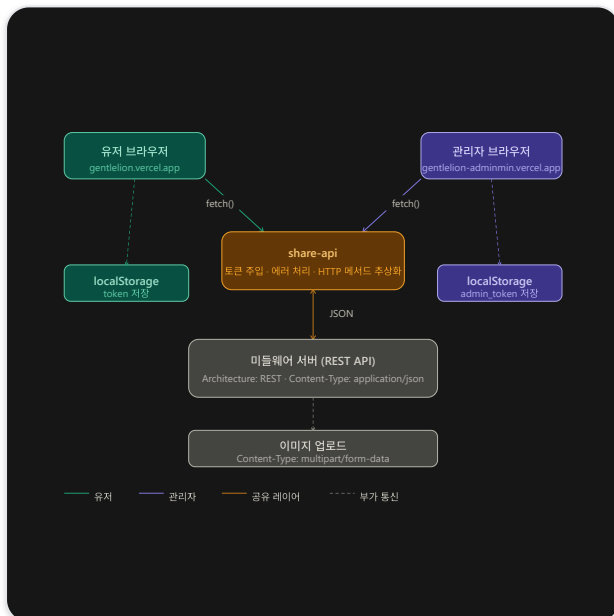
제가 맡은 검증 범위

5개 핵심 흐름

회원가입 · 로그인 · 장바구니 · 주문 · 프로필

- Playwright 설치와 유저·관리자 테스트 폴더 구조 설계
- 팀 전체 결과물은 **20개 spec**, 개인 작성 범위는 5개 흐름으로 구분

공유 API 중심 통신 구조



동적 헤더 로드 뒤 사라지던 메뉴 애니메이션

헤더를 비동기로 DOM에 주입하기 전에 메뉴 초기화 코드가 실행되어 대상 요소를 찾지 못했습니다. 헤더 로드 완료 후 초기화하도록 실행 순서를 조정하고, 모바일 메뉴 생성 책임을 한 곳으로 모았습니다.

로컬과 배포 환경의 경로 차이

Vercel에서 동적 컴포넌트 경로가 달라 유저·관리자 앱 일부가 로드되지 않았습니다. 현재 origin을 기준으로 경로를 계산하고 각 패키지의 Vite 설정을 정리해 분리 배포 환경에서 동일하게 동작하도록 수정했습니다.

테스트 코드 확인 [↗](#)

끝

"끝은 새로운 시작입니다."

WEB FRONTEND DEVELOPER

김도혁 · Kim Do Hyuk

PORTFOLIO · 2026